// Day 4 of 10

Day 4

10 Java Tips

Developers make these every day.
Don't be one of them.

→ Swipe through all 10

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #01

Use Optional Properly

Stop returning null — causes NPEs

```
// Bad
public User findUser(Long id){
  return null; // caller forgets check
}

// Good
public Optional findUser(Long id){
  return repo.findById(id);
}
findUser(1L).orElseThrow();
```

💡 Optional forces callers to handle absence

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #02

Use var for Local Variables

Reduces boilerplate, cleaner code

```
// Bad
HashMap> map
  = new HashMap>();

// Good
var map = new HashMap>();
var users = repo.findAll();
// Java 10+ feature
```

💡 var works great for local variables

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #03

Use Streams Correctly

Cleaner than for-each loops

```
// Bad
List names = new ArrayList<>();
for(User u : users){
  if(u.isActive())
    names.add(u.getName());
}

// Good
var names = users.stream()
  .filter(User::isActive)
  .map(User::getName)
  .collect(toList());
```

💡 Streams = readable, functional style

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #04

StringBuilder for Concatenation

+ in loops creates many objects

```
// Bad
String s = "";
for(int i = 0; i < 1000; i++)
  s += i; // new object each time!

// Good
var sb = new StringBuilder();
for(int i = 0; i < 1000; i++)
  sb.append(i);
String s = sb.toString();
```

💡 Use StringBuilder in loop concatenation

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #05

Records for Data Classes

Replaces verbose POJOs/DTOs

```
// Bad
public class UserDTO {
  private String name;
  private String email;
  // + getters, setters, equals ...
}

// Good
public record UserDTO(
  String name,
  String email
) {} // Java 14+ — that's it!
```

💡 Records auto-generate everything

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #06

Switch Expressions

Cleaner than old switch statements

```
// Bad
String label;
switch(status){
  case 1: label="Active"; break;
  case 2: label="Inactive"; break;
  default: label="Unknown";
}

// Good
String label = switch(status){
  case 1 → "Active";
  case 2 → "Inactive";
```

💡 Switch expressions — Java 14+

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #07

Immutable Collections

Prevent accidental modification

```
// Bad
List list = new ArrayList<>();
list.add("a");
// Anyone can modify this!

// Good
List list = List.of("a","b","c");
Map map = Map.of("a",1);
// Immutable — safe to share
```

💡 List.of() / Map.of() since Java 9

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #08

Try-with-Resources

Always close your resources

```
// Bad
FileReader fr = new FileReader("f.txt");
// Exception? fr never closed!

// Good
try(var fr = new FileReader("f.txt")){
  // fr auto-closed after block
}
```

💡 Implements AutoCloseable = use try-with

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #09

Avoid Raw Types

Raw types skip compile-time checks

```
// Bad
List list = new ArrayList();
list.add("text");
list.add(123); // No compile error!

// Good
List list = new ArrayList<>();
list.add("text");
// list.add(123); // Compile error!
```

💡 Always use generics — no raw types

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #10

Comparator.comparing

Cleaner sorting for objects

```
// Bad
users.sort((a,b) → {
  return a.getName().compareTo(b.getName());
});

// Good
users.sort(
  Comparator.comparing(User::getName)
    .thenComparing(User::getAge)
);
```

💡 Chain comparators cleanly

Shubham Bhati
linkedin.com/in/bhatishubham

```
done = true
```

Follow for more tips!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham